

Inheritance: Building New Classes from Old Ones

Quiz 04 — ANSWER KEY · 20 points

array[0] Predict the Output

3 pts

OUTPUT

B M T | ~T ~M ~B

Constructors run base-first, all the way down the chain (B M T). Destructors run in the exact reverse order when t leaves scope at the closing brace of main , which is after the I prints.

array[1] Who Can See What?

5 pts

LINE	COMPILES?	REASON
A	N	balance has no label, so it is private . Private members are inherited (they exist inside every Savings) but only Account 's own code can touch them.
B	Y	owner is protected , so derived-class member functions can use it.
C	Y	getBalance() is public in Account and stays public through public inheritance.
D	N	protected only opens access to derived classes. main() is outside code, so it is blocked just as if owner were private.

FIX FOR LINE A

```
class Account {
protected:
    double balance;    // moved under protected:
    string owner;
    ...
}
```

array[2] Override, Overload, or Neither?

7 pts · 1 each

ITEM	ANSWER
Animal::speak() / Dog::speak()	OVERRIDDEN: same signature, redefined in the derived class
Dog::speak() / Dog::speak(int)	OVERLOADED: same name, different parameter lists, same class
Animal::eat(string) / Dog::fetch()	NEITHER: different names, unrelated functions
a.speak();	...
d.speak();	Woof
d.eat("bone");	eats bone (inherited unchanged from Animal)
(c) inside Dog::speak()	Animal::speak();

```

class Manager : public Employee {
    int reports;
public:
    Manager(string n, double s, int r) : Employee(n, s), reports(r) {}

    void print() {
        Employee::print();
        cout << "Reports: " << reports << "\n";
    }
};

```

Manager m("Kim", 90000, 4); m.print(); prints Kim: \$90000 then Reports: 4.

GRADING NOTES

What counts

array[0]	1 pt for B M T, 1 pt for ~T ~M ~B (reverse order), 1 pt for the destructors appearing <i>after</i> the . Destructors in constructor order (~B ~M ~T) loses the second point.
array[1]	1 pt per line (Y/N plus a reason that names the access level). The fix (1 pt) is protected . Also accept adding a protected (or public) setter/deposit method to Account and calling it in Line A. Making balance public or making Savings a friend earns 0, because the first breaks the stated rule and the second dodges the point of the question.
array[1] watch	"A fails because balance isn't inherited" is wrong . It is inherited but not accessible (W05 Q2). Deduct the reason point.
array[2] note	Without virtual this is technically <i>hiding/redefinition</i> ; W05 calls it OVERRIDDEN , so accept either term. A student who points out the difference gets a nod; it's next week's topic (W06 Polymorphism).
array[3]	1 pt each: : public Employee ; int reports private ; Employee (n , s) in the initializer list; Employee::print() call; prints reports . Assigning name = n ; salary = s ; in the constructor body instead of chaining does not compile (Employee has no default constructor), so no chaining point. Forgetting Employee:: (so print() calls itself forever) loses the call point. Inheritance with no public keyword (private inheritance) loses the first point.
Timing	Roughly 3 / 4 / 3 / 5 minutes. If students run short, array[3] is the one to allow pseudo-C++ on.